

Brainary CLI使用说明

Brainary CLI 使用说明

本文说明如何通过命令行启动和使用 Brainary，包括全局参数、子命令、自动化调用和会话管理。

```
(base) └─bixi@MacBook-Air ~
└─$ brainary --help
Brainary CLI

If no subcommand is specified, options will be forwarded to the interactive CLI.

Usage: brainary [OPTIONS] [PROMPT]
       brainary [OPTIONS] <COMMAND> [ARGS]

Commands:
  exec      Run Brainary non-interactively [aliases: e]
  review    Run a code review non-interactively
  mcp        Manage external MCP servers for Brainary
  plugin     Manage Brainary plugins
  mcp-server Start Brainary as an MCP server (stdio)
  app-server [experimental] Run the app server or related tooling
  remote-control [experimental] Manage the app-server daemon with remote control enabled
  app        Launch the Desktop app (opens the app installer if missing)
  completion Generate shell completion scripts
  doctor     Diagnose local Brainary installation, config, auth, and runtime health
  sandbox    Run commands within a Brainary-provided sandbox
  debug      Debugging tools
  apply      Apply the latest diff produced by Brainary as a `git apply` to your local working tree [aliases: a]
  resume     Resume a previous interactive session (picker by default; use --last to continue the most recent)
  archive    Archive a saved session by id or session name
  delete     Permanently delete a saved session by id or session name
  unarchive  Unarchive a saved session by id or session name
  fork       Fork a previous interactive session (picker by default; use --last to fork the most recent)
  cloud      [EXPERIMENTAL] Browse cloud tasks and apply changes locally
  exec-server [EXPERIMENTAL] Run the standalone exec-server service
  features   Inspect feature flags
  help       Print this message or the help of the given subcommand(s)

Arguments:
  [PROMPT]
    Optional user prompt to start the session
```

图1：使用说明截图

快速开始

```
代码块

1  # 查看帮助和版本
```

```
2  brainary --help
3  brainary --version
4
5  # 指定项目目录并启动 Brainary
6  brainary -C /path/to/project
7
8  # 非交互执行一次任务
9  brainary exec "运行测试并总结失败原因"
10
11 # 审查当前未提交修改
12 brainary review --uncommitted
```

命令写法

命令格式中的符号含义如下：

写法	含义
<value>	必填参数，例如 <SESSION>
[value]	可选参数，例如 [PROMPT]
...	可以提供多个值
-x / --xxx	短参数与长参数，两种写法作用相同
brainary <subcommand> --help	查看该子命令的完整参数

不带子命令运行 `brainary` 时，会进入 TUI。带 `exec`、`review` 等子命令时，会执行对应的 CLI 功能。

全局参数

以下参数用于启动 TUI，也有一部分可以用于 `resume`、`fork` 等子命令。

参数	类型或取值	说明
[PROMPT]	字符串	启动会话时直接提交的任务描述
-C, --cd	目录路径	将指定目录作为 Brainary 的工作目录
--add-dir	目录路径，可重复	额外允许写入的目录
-m, --model	模型名称	指定本次会话使用的模型

<code>-i, --image</code>	文件路径，可重复	将一个或多个图片附加到初始任务
<code>-p, --profile</code>	配置名称	在基础配置上叠加 <code>\$CODEX_HOME/<name>.config.toml</code>
<code>-c, --config</code>	<code>key=value</code> ，可重复	临时覆盖配置项；嵌套项使用点路径，值按 TOML 解析
<code>--strict-config</code>	boolean flag	配置中出现当前版本无法识别的字段时直接报错
<code>--enable</code>	功能名称，可重复	为本次运行启用功能
<code>--disable</code>	功能名称，可重复	为本次运行禁用功能
<code>-s, --sandbox</code>	<code>read-only</code> 、 <code>workspace-write</code> 、 <code>danger-full-access</code>	设置模型生成命令的沙箱级别
<code>-a, --ask-for-approval</code>	<code>untrusted</code> 、 <code>on-request</code> 、 <code>never</code>	设置命令执行的审批策略
<code>--approve-for-me</code>	boolean flag	使用 <code>workspace-write</code> 沙箱，并把审批请求交给自动审查；适合希望减少人工确认但仍保留沙箱的任务
<code>--search</code>	boolean flag	启用实时网页搜索能力
<code>--oss</code>	boolean flag	使用开源模型提供方
<code>--local-provider</code>	<code>lmstudio</code> 、 <code>ollama</code>	指定本地模型提供方，通常与 <code>--oss</code> 一起使用
<code>--remote</code>	<code>ws://</code> 、 <code>wss://</code> 、 <code>unix://</code> 地址	连接远程 app-server
<code>--remote-auth-token-env</code>	环境变量名	从指定环境变量读取远程连接的 Bearer Token
<code>--no-alt-screen</code>	boolean flag	使用内联模式运行 TUI，保留终端滚动历史
<code>-h, --help</code>	boolean flag	显示帮助
<code>-V, --version</code>	boolean flag	显示版本

命令总览

普通用户命令

命令	说明
<code>brainary</code>	启动交互式 TUI
<code>brainary exec</code> / <code>brainary e</code>	非交互执行任务，适合脚本和 CI
<code>brainary review</code>	非交互审查代码修改
<code>brainary resume</code>	恢复已有会话
<code>brainary fork</code>	从已有会话创建分支会话
<code>brainary archive</code>	归档指定会话
<code>brainary unarchive</code>	恢复已归档会话
<code>brainary delete</code>	永久删除指定会话
<code>brainary mcp</code>	管理外部 MCP 服务
<code>brainary plugin</code>	管理 Brainary 插件和插件市场
<code>brainary completion</code>	生成 Shell 自动补全脚本
<code>brainary doctor</code>	检查安装、配置、认证、运行环境、Git、终端、MCP 和会话状态
<code>brainary sandbox</code>	在 Brainary 提供的沙箱中执行命令
<code>brainary features</code>	查看或修改功能开关

开发与集成命令

这些命令面向客户端开发、协议调试、服务部署或自动化集成，普通 TUI 用户通常不需要直接运行。

命令	说明
<code>brainary mcp-server</code>	通过标准输入输出将 Brainary 启动为 MCP 服务
<code>brainary app-server</code>	实验性接口：启动 app-server，并提供协议 Schema 和绑定生成工具
<code>brainary exec-server</code>	实验性接口：在本地启动独立 exec-server，可通过 stdio 或 WebSocket 接入
<code>brainary debug</code>	

Brainary 项目扩展能力

当前仓库相对上游最明确的扩展不是命令改名，而是 PoA 执行链：

- `brainary exec --poa <PATH>`：从 CLI 提交包目录或 `.poa` 文件。
- `thread/codeMode/exec`：app-server 提供的实验性 JSON-RPC 方法，请求携带线程 ID 和 PoA 包，可以直接运行一个 code-mode cell，不需要先让模型生成该程序。
- PoA 自带能力：包可以携带资源及 `manifest.toml` 声明的 MCP 服务；服务端负责校验、解包，等待这些服务就绪后仅为对应线程运行它们。

这些是开发者集成能力，不应与普通对话命令混为一谈。PoA 中的控制程序本身直接执行；如果程序调用 Agent，相关 Agent 调用仍然会产生模型请求。

当前关闭的入口

入口	当前边界
<code>brainary login</code> / <code>brainary logout</code>	普通账号登录和退出暂不可用； <code>brainary mcp login/logout</code> 仅管理第三方 MCP OAuth
<code>brainary update</code>	暂不可用，Brainary 自有安装与更新链路尚未启用
<code>brainary app</code> / <code>brainary apply</code> / <code>brainary cloud</code>	不属于当前 Brainary 公共能力，已从帮助中隐藏并拒绝执行
<code>brainary remote-control</code> / <code>brainary app-server daemon</code> / <code>brainary app-server proxy</code> / <code>brainary app-server --remote-control</code>	依赖当前未启用的托管 app-server，已从帮助中隐藏并拒绝执行；普通 <code>app-server</code> 也不会启用旧的持久化远程控制设置
<code>brainary exec-server --remote</code> 及其远程注册参数	依赖尚未接入 Brainary 的托管环境注册服务，已从帮助中隐藏并拒绝执行；本地 <code>exec-server --listen ...</code> 保持可用
<code>apps</code> / <code>in_app_updates</code> 功能开关	上游连接器与应用内更新链路未接入 Brainary，默认关闭且不能通过普通 CLI 配置启用
上游内置插件市场	不自动加载或展示，也不允许从已知上游市场新增、安装或升级；已有旧插件和市场配置仍允许删除清理；用户自行配置的本地或 Git 市场不受影响

命令详情

启动 Brainary

代码块

```
1 brainary [全局参数] [PROMPT]
```

示例：

代码块

```
1 # 在当前项目启动
2 brainary
3
4 # 指定目录和模型
5 brainary -C /path/to/project -m <model-name>
6
7 # 附带图片说明问题
8 brainary -i screenshot.png "分析这个界面问题"
9
10 # 启用网页搜索
11 brainary --search "调研这个依赖的最新版本"
```

非交互执行： `exec`

代码块

```
1 brainary exec [OPTIONS] [PROMPT]
2 brainary exec resume [OPTIONS]
3 brainary exec review [OPTIONS]
```

如果不在参数中提供任务，或将任务写成 `-`，Brainary 会从标准输入读取内容。若同时提供任务参数和管道输入，管道内容会作为 `<stdin>` 块追加。

代码块

```
1 # 一次性执行
2 brainary exec "检查格式并报告问题"
3
4 # 从标准输入传入任务
5 printf '%s\n' "总结当前修改" | brainary exec -
6
7 # 输出 JSONL 事件，供程序处理
8 brainary exec --json "运行测试"
9
```

```
10 # 将最后一条模型消息写入文件
11 brainary exec -o result.txt "生成发布说明"
12
13 # 不保存本次会话文件
14 brainary exec --ephemeral "解释这个函数"
```

常用的 `exec` 专用参数：

参数	类型或取值	说明
<code>--skip-git-repo-check</code>	boolean flag	允许在非 Git 仓库目录中运行
<code>--ephemeral</code>	boolean flag	不在磁盘上保存本次会话
<code>--ignore-user-config</code>	boolean flag	不读取用户配置；认证仍使用 <code>CODEX_HOME</code>
<code>--ignore-rules</code>	boolean flag	不读取用户或项目的 <code>execpolicy.rules</code> 文件
<code>--output-schema</code>	文件路径	用 JSON Schema 约束最终回复结构
<code>--json</code>	boolean flag	以 JSONL 输出执行事件
<code>-o, --output-last-message</code>	文件路径	将最后一条模型消息写入文件
<code>--color</code>	<code>always</code> 、 <code>never</code> 、 <code>auto</code>	设置标准输出的颜色模式
<code>--poa</code>	目录路径或 <code>.poa</code> 文件	运行 PoA 包；这是当前 Brainary 项目的扩展能力

运行 PoA: `exec --poa`

PoA（Program of Agent）是当前 Brainary 项目的扩展能力。它把程序逻辑、`manifest.toml`、资源以及可选的内置 MCP 服务封装成一个可执行包，由 code-mode 后端运行。

代码块

```
1 # 直接运行包目录；Brainary 会在发送前现场打包
2 brainary exec --poa workflow-demos/poas/00_echo
3
4 # 运行已经打包好的单文件
5 brainary exec --poa ./00_echo.poa
```

使用条件和限制：

- 包目录根部必须包含有效的 `manifest.toml`；`.poa` 文件是相同目录结构的打包形式。
- 当前命令只负责读取或打包文件并把包内容交给 code-mode 后端；解包、启动清单声明的 MCP 服务和实际执行发生在后端。
- `--poa` 不能同时提供普通 `[PROMPT]`，也不能与 `exec resume` 或 `exec review` 子命令组合。
- PoA 一次运行一个 code-mode cell，不等同于普通的模型对话循环。

预期结果：Brainary 校验并提交 PoA 包，打印该 cell 的执行输出；路径、清单或包格式无效时会在执行前报错。

代码审查： `review`

代码块

```
1 brainary review [OPTIONS] [PROMPT]
```

参数	类型或取值	说明
<code>--uncommitted</code>	boolean flag	审查已暂存、未暂存和未跟踪的修改
<code>--base</code>	分支名	审查当前分支相对指定基础分支的修改
<code>--commit</code>	Git 提交 SHA	审查指定提交引入的修改
<code>--title</code>	字符串	为审查摘要提供提交标题；仅与 <code>--commit</code> 一起使用
<code>[PROMPT]</code>	字符串或 <code>-</code>	提供自定义审查要求；使用 <code>-</code> 时从标准输入读取

示例：

代码块

```
1 brainary review --uncommitted
2 brainary review --base main "重点检查兼容性问题"
3 brainary review --commit <commit-sha>
```

会话管理

参数或子命令	类型或取值	说明
<code>brainary resume</code>	命令	打开会话选择器
<code>brainary resume --last</code>	boolean flag	直接恢复当前目录中最近一次会话
<code>brainary resume <SESSION></code>	会话 UUID 或名称	恢复指定会话；UUID 优先于名称匹配
<code>brainary resume --all</code>	boolean flag	选择最近会话时不按当前目录过滤
<code>brainary fork</code>	命令	打开会话选择器并创建分支会话
<code>brainary fork --last</code>	boolean flag	从当前目录中最近一次会话创建分支会话
<code>brainary archive <SESSION></code>	会话 UUID 或名称	归档会话但保留记录
<code>brainary unarchive <SESSION></code>	会话 UUID 或名称	恢复已归档会话
<code>brainary delete <SESSION></code>	会话 UUID 或名称	永久删除会话；名称仍需要交互确认
<code>brainary delete <UUID> --force</code>	会话 UUID	跳过确认并永久删除指定会话

`brainary delete` 是不可恢复操作。`--force` 会跳过确认，而且此时会话参数必须是 UUID。

MCP 服务管理

代码块

```
1 brainary mcp <COMMAND>
```

参数或子命令	类型或取值	说明
<code>list</code>	可选 <code>--json</code>	列出已配置的 MCP 服务； <code>--json</code> 输出机器可读结果
<code>get <NAME></code>	服务名称，可选 <code>--json</code>	查看指定 MCP 服务配置

<code>add <NAME></code>	<code>-- <COMMAND...></code> 或 <code>--url <URL></code>	添加 stdio 或 HTTP MCP 服务
<code>remove <NAME></code>	服务名称	移除 MCP 服务配置
<code>login <NAME></code>	服务名称, 可选 OAuth scope	为支持 OAuth 的 HTTP MCP 服务授权
<code>logout <NAME></code>	服务名称	清除外部 MCP 服务的 OAuth 授权

这里的 `mcp login/logout` 只管理第三方 MCP 服务授权, 不是 Brainary 账号登录。当前普通用户 CLI 不提供顶层 `brainary login` 或 `brainary logout`。

插件管理

CLI 插件命令只展示本地或用户自行配置的插件市场, 不自动加载上游内置市场。不能从已知上游市场新增、安装或升级插件, 不带市场名批量升级时也会跳过这些市场。为避免旧配置无法清理, `plugin remove` 和 `plugin marketplace remove` 仍允许删除已经存在的上游插件或市场。TUI 中的插件目录入口当前不公开, `@` 提及候选只查询并保留用户本地市场。

参数或子命令	类型或取值	说明
<code>list</code>	可选市场名和 JSON 输出	列出已安装或指定市场中可发现的插件
<code>add <PLUGIN></code>	插件名, 可选市场名	从已配置的市场安装插件
<code>remove <PLUGIN></code>	插件名	移除插件及其本地缓存配置
<code>marketplace <COMMAND></code>	<code>add</code> 、 <code>list</code> 、 <code>upgrade</code> 、 <code>remove</code>	添加、列出、升级或移除插件市场

环境诊断: `doctor`

代码块

```
1 brainary doctor
2 brainary doctor --summary
3 brainary doctor --json
```

参数	类型或取值	说明
<code>--summary</code>	boolean flag	只显示分组检查结果和最终统计

<code>--json</code>	boolean flag	输出已脱敏的机器可读报告
<code>--all</code>	boolean flag	展开详细输出中的长列表
<code>--no-color</code>	boolean flag	关闭 ANSI 颜色
<code>--ascii</code>	boolean flag	使用 ASCII 状态标记和分隔符

`doctor` 用于安装异常、配置加载失败、认证失效、终端显示问题或服务连接问题。报告会检查安装来源与可执行文件、配置、已有认证状态、运行时、Git、终端、状态目录、MCP、沙箱、网络、app-server 和会话索引等项目。

代码块

```
1  # 日常排障：查看完整的人类可读报告
2  brainary doctor
3
4  # 开会或提 issue 前：生成已脱敏的机器可读报告
5  brainary doctor --json > brainary-doctor.json
```

预期结果：每项检查显示正常、警告或失败，并给出汇总；`--json` 适合交给排障工具处理，但分享前仍应人工检查内容。

开发与集成命令

代码块

```
1  # 通过 stdio 启动 MCP 服务
2  brainary mcp-server
3
4  # 通过 stdio 启动 app-server
5  brainary app-server --listen stdio://
6
7  # 启动独立 exec-server
8  brainary exec-server --listen stdio://
```

`app-server` 和 `exec-server` 是实验性接口，适合客户端开发、自动化或协议调试。部署到非本机地址前必须另外设计传输安全、鉴权和生命周期管理，不能按普通用户命令直接对公网开放。

Shell 自动补全

代码块

```
1  brainary completion <SHELL>
```

支持 `bash`、`elvish`、`fish`、`powershell` 和 `zsh`。例如，为 `zsh` 生成补全脚本：

代码块

```
1 brainary completion zsh > _brainary
```

具体安装位置取决于本机 Shell 的补全目录设置。

功能开关

代码块

```
1 brainary features list
2 brainary features enable <FEATURE>
3 brainary features disable <FEATURE>
```

也可以只对本次运行临时启用或禁用：

代码块

```
1 brainary --enable <FEATURE>
2 brainary --disable <FEATURE>
```

常见操作流程

在自动化中运行

代码块

```
1 brainary exec --json -C /path/to/project \
2 "运行项目测试，只报告失败项和可能原因" > events.jsonl
```

自动化环境中仍建议保留沙箱。只有外部环境已经完成可靠隔离时，才考虑绕过审批和沙箱。

恢复或整理会话

代码块

```
1 # 恢复最近一次会话
2 brainary resume --last
3
4 # 查看所有目录中的会话
```

```
5 brainary resume --all
6
7 # 归档不再活跃的会话
8 brainary archive <会话名称或UUID>
9
10 # 恢复归档会话
11 brainary unarchive <会话名称或UUID>
```

参数组合与安全提示

- 日常本地开发优先使用 `--sandbox workspace-write --ask-for-approval on-request`，让 Brainary 可以修改工作区，但在需要额外权限时暂停确认。
- 需要访问工作区以外的目录时，优先使用 `--add-dir <PATH>` 明确增加目录，不要直接切换到 `danger-full-access`。
- `--dangerously-bypass-approvals-and-sandbox` 会同时关闭审批和沙箱，只能用于外部已经提供可靠隔离的环境。
- `--dangerously-bypass-hook-trust` 会跳过 Hook 的持久化信任检查，只适用于已经独立验证 Hook 来源的自动化环境。
- `-a never` 表示执行过程中不再请求人工确认；它不等同于关闭沙箱，实际访问范围仍由 `--sandbox` 决定。
- CI 中可组合 `--json` 与 `--output-last-message <FILE>`，分别保存过程事件和最终自然语言结果。

配置与兼容路径

当前版本仍沿用上游兼容配置路径和环境变量：

标识	用途
<code>~/.codex/config.toml</code>	默认用户配置文件
<code>CODEX_HOME</code>	覆盖配置、认证及会话数据目录
<code>\$CODEX_HOME/<name>.config.toml</code>	<code>--profile <name></code> 对应的叠加配置文件
<code>\$CODEX_HOME/auth.json</code>	已有兼容认证文件；当前版本继续读取，但不提供登录界面来新建或更新它
<code>OPENAI_API_KEY</code>	默认兼容 Provider 的 API Key 环境变量；名称为兼容标识，当前不改名

这些名称属于存储和配置兼容标识。保留它们可以继续读取已有配置与会话，不代表用户启动命令仍是 `codex`；当前 CLI 入口是 `brainary`。

当前版本不提供顶层 `brainary login`、`brainary logout` 或 `brainary update`。如果后续接入新的账户或更新体系，应以对应版本的 `brainary --help` 为准。

获取帮助

代码块

```
1  brainary --help
2  brainary exec --help
3  brainary review --help
4  brainary mcp --help
5  brainary plugin --help
```